RAG and Agents

Retrieval algorithms, context optimization, tool planning, execution, evaluation, and memory

RAG obtains useful information; agents choose and execute actions toward goals. Both depend on explicit context, trustworthy tools, observable intermediate states, and evaluation of the final task.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 253–306 / PDF p. 277. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 RAG Architecture and Retrieval Algorithms

Source: book pp. 253–267 / PDF p. 277

Why retrieval, and what gets indexed?

RAG retrieves query-specific information from external memory and supplies it to a generator. It supports current/private facts, large knowledge collections, and source-grounded responses without embedding all facts in weights. Retrieval augments context; it does not guarantee the generator uses it correctly.

Indexing path: ingest and parse documents, preserve metadata/permissions, chunk, create indexes. **Query path:** interpret query, retrieve candidate chunks, optionally filter/rerank, construct context, generate, validate. For a small knowledge base, placing all relevant content in a long context can be a useful baseline; compare actual accuracy and cost rather than assuming RAG is mandatory.

Term-based retrieval: TF-IDF and BM25

An inverted index maps terms to documents containing them. Term frequency captures occurrence; inverse document frequency downweights ubiquitous terms. BM25 adds term-frequency saturation and document-length normalization and remains a strong, inexpensive baseline.

Term retrieval is fast, transparent, and good for identifiers, exact part numbers, and distinctive names. It can miss synonyms or ambiguous wording. Tokenization, stop words, stemming, language, and query normalization affect results. Sparse versus dense is a representation distinction; learned sparse systems mean it is not identical to lexical versus semantic.

Embedding-based retrieval and vector search

Encode chunks and queries into compatible vector spaces, then search for nearby chunk vectors. Semantic retrieval can match paraphrases and bridge lexical variation, but requires relevant embeddings and carries indexing/query costs.

Brute-force nearest neighbors are expensive at scale. Approximate nearest-neighbor (ANN) search trades exactness for speed. The book discusses locality-sensitive hashing, tree/partition approaches such as inverted-file indexes, product quantization, and graph search such as HNSW. Storing vectors is easy; maintaining efficient search, filtering, updates, and consistency is the harder database problem.

Evaluate retrieval and combine algorithms

Context precision is the relevant fraction of retrieved items; recall is the recovered fraction of all relevant items. Recall is harder to label because it requires knowing missing relevant items. MRR, MAP, and NDCG assess rankings; ANN recall measures recovery relative to exact nearest neighbors and is different from relevance recall.

Also measure indexing time, query latency/QPS, storage, update speed, and access controls. Hybrid search combines lexical and semantic strengths. Reciprocal-rank fusion combines rankings without requiring comparable raw scores:

> $\text{RRF score}(d) = \Sigma_{\text{retrievers}}\ 1 / (k + \text{rank}(d))$

Rerank candidates with a more expensive model only when the quality gain justifies latency.

---

02 Retrieval Optimization and RAG Beyond Text

Source: book pp. 267–275 / PDF p. 291

Chunking strategy

Choose boundaries by tokens, sentences, paragraphs, sections, layout, or meaning. Small chunks allow precise matches and more diverse evidence; overly small chunks lose context. Large chunks preserve relationships but can waste context budget and dilute relevance.

Overlap, parent-document links, titles, and metadata preserve meaning across boundaries. Evaluate table/code handling and the tokenizer assumptions. Rechunking or changing embeddings can require expensive reindexing. There is no universally best chunk size; tune against representative questions and storage/latency constraints.

Reranking, query rewriting, contextual retrieval

- **Reranking:** apply a stronger relevance model to initial candidates and select evidence within the generator's budget.
- **Query rewriting:** resolve conversational references, normalize wording, expand terms, or decompose a question. Do not invent missing entity relationships.
- **Contextual retrieval:** enrich chunks with relevant document-level context, metadata, or generated questions before indexing so isolated fragments become findable.

Search ranking determines which evidence is included; ordering within the prompt also affects use, especially with long-context position effects. Test both separately.

Multimodal and tabular RAG

Multimodal retrieval can index captions/metadata or use joint embeddings to retrieve images, text, and other content. The generator must be able to consume the retrieved modality or a faithful representation of it.

For structured data, a typical flow is schema/table selection, text-to-SQL generation, safe query execution, and answer generation from results. Validate SQL, restrict permissions, cap costs, and retain query/results for audit. Numerical aggregations belong in reliable computation tools rather than free-form guessing.

---

03 Agents: Overview, Tools, and Planning

Source: book pp. 275-298 / PDF p. 299

Agent, environment, goal, and tool inventory

An agent perceives an environment and selects actions toward a goal under constraints. A foundation model often supplies the planner, but the system also needs tool execution, state, evaluation, and stopping rules. A fixed retrieval pipeline and a dynamic agent differ in who selects actions and how control flow adapts.

Tools support **knowledge augmentation** (search/database reads), **capability extension** (calculation, code, conversion), and **write actions** (messages, orders, record changes). More tools increase opportunity and selection difficulty. Expose a minimal useful inventory with clear names, schemas, descriptions, and error behavior.

Planning and function calling

Plan against both the goal and constraints such as budget, deadline, and permissions. A plan may cover the whole task or only the next step. Natural-language plans can be translated to executable calls; direct structured tool plans skip that translator but must match valid API syntax.

The model proposes a function name and arguments; trusted application code checks and executes it, then returns an observation. A valid function name or JSON schema does not prove correct arguments or authorization. Validate preconditions and results, handle unavailable tools, and reject infeasible/out-of-scope requests early.

Control flow, reflection, and correction

Workflows may be sequential, parallel, conditional, or iterative. Some steps depend on earlier observations, so blindly executing a full initial plan can fail. The book treats general LLM planning ability as an open research question, not an established guarantee.

ReAct interleaves reasoning/action with observations. **Reflexion** separates outcome evaluation and analysis of what went wrong so later attempts can use the lesson. Reflection can occur before a plan, after a tool call, or after completion. It must lead to grounded correction, not merely additional confident text. Limit retries, steps, and cost.

Tool selection and extensibility

Use ablations to see which tools actually improve success. Remove tools that add confusion without helping; redesign descriptions or APIs that models frequently misuse. Monitor transitions between tools to discover useful combinations or reusable skills.

Check whether a framework supports your control flows, state persistence, new tools, and observability. A second agent can evaluate a plan, but more agents also create communication, coordination, latency, and debugging costs.

---

04 Agent Failure Modes and Evaluation

Source: book pp. 298–300 / PDF p. 322

Separate planning, tools, and efficiency

Planning failures: invalid steps, wrong order, ignored constraints, incorrect assumptions, or failure to stop. **Tool failures:** wrong selection, arguments, permissions, unavailable capability, API exceptions, incorrect tool results, or errors translating a high-level plan.

Test tools independently and inspect each call and observation. Measure actual end-to-end completion, constraint satisfaction, latency, cost, number of calls, redundant actions, and recovery. A successful intermediate plan is not a completed task, and success after a deadline may be failure. Run state-changing evaluations in a sandbox with controlled resources.

---

05 Memory

Source: book pp. 300–306 / PDF p. 324

Three memory mechanisms

Internal knowledge: parameters learned during training. **Short-term memory:** immediately available context. **Long-term memory:** externally stored information retrieved when needed.

Put durable general behavior in weights when appropriate, infrequently needed facts in retrievable storage, and current-task state in context. External memory can preserve structured records, reduce repetitive input, support personalization, and calibrate consistency, but needs access, update, correction, and deletion policies.

Managing a finite context

Conversation history, tools, evidence, plans, and reflections compete for the same window. FIFO or last-N retention is easy but may delete the original goal. Summarization saves space while risking omission; removing redundant/less relevant material and retaining key facts can be better.

Combine retained instructions, rolling summaries, structured task state, and retrieval of older relevant records. Check that memories remain accurate and authorized; remembered model errors can otherwise become persistent false "facts." Memory quality must be evaluated by downstream task success, not simply recall quantity.

Revision Check

1. Why is BM25 a useful baseline even when embeddings are available?
2. How do relevance recall and ANN recall differ?
3. Which chunking failure could hide an answer across two pages?
4. What validates a tool call before execution?
5. How could FIFO memory destroy the task goal?

Retrieve evidence with measurable relevance, execute tools through trusted controls, and evaluate complete tasks. Add dynamic planning only when it earns its complexity.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.